CS-338 Intro to the Theory of Computation

- Lecture #9
- **Decidability**

- Prof. Pietro S. Oliveto
  Department of Computer Science and Engineering
  Southern University of Science and Technology (SUSTech)
- olivetop@sustech.edu.cn
  https://faculty.sustech.edu.cn/olivetop
- Office: Room 113

Overview

Last time:

- Equivalence of variants of the Turing machine model
  a. Multi-tape TMs
  b. Nondeterministic TMs
  c. Enumerators
- Church-Turing Thesis
- Notation for encodings and TMs

Today: (Sipser §4.1)

- Decision procedures for automata and grammars

TMs and Encodings – review

A TM has 3 possible outcomes for each input w:

1. *Accept* w (enter q_{acc})
2. *Reject* w by halting (enter q_{rej})
3. *Reject* w by looping (running forever)

A is T-recognizable if $A = L(M)$ for some TM M.

A is T-decidable if $A = L(M)$ for some TM decider M.

halts on all inputs

$\langle O_1, O_2, \ldots, O_k \rangle$ encodes objects $O_1, O_2, \ldots, O_k$ as a single string.

Notation for writing a TM M is

$M =$ "On input w

[English description of the algorithm]"

TMs and Algorithms

- The Church-Turing thesis allowed us to define the notion of **algorithm** in terms of Turing machines
- Our objective is to explore the limits of algorithmic **solvability**
- The notion of solvability is common in computer science
- The notion of **unsolvability** is less common
- Knowing when a problem is algorithmically unsolvable allows you to realise that a problem needs to be **simplified** or **altered** before you can find an algorithmic solution
- Like any other tool computers have **capabilities** and **limitations** that have to be appreaciated to be used well.

Acceptance Problem for DFAs

- The **acceptance problem** for DFAs is that of testing whether a deterministic finite automaton (DFA) accepts a given string.
- For our convenience we can express such algorithmic problem as a language.

Let $A_{\text{DFA}} = \{\langle B, w\rangle | \ B \text{ is a DFA and } B \text{ accepts } w\}$

This language contains all possible DFAs paired with the strings that the DFAs accept.

Is it decidable?

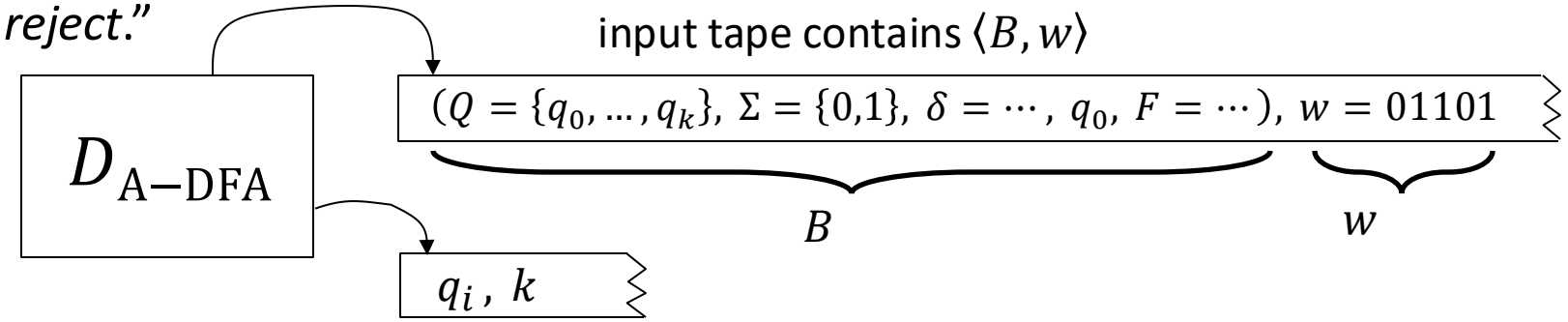
Theorem: A_{DFA} is decidable

Proof: Give TM $D_{\text{A–DFA}}$ that decides A_{DFA}.

$\boldsymbol{D_{\text{A–DFA}}}$ = "On input s

1. Check that s has the form $\langle B, w\rangle$ where B is a DFA and w is a string; *reject* if not.
2. Simulate the computation of B on w.
3. If B ends in an accept state then *accept*. If not then *reject*."

Shorthand:
On input $\langle B, w\rangle$



Acceptance Problem for NFAs

- The **acceptance problem** for NFAs is that of testing whether a non-deterministic finite automaton accepts a given string.

Let $A_{\text{NFA}} = \{\langle B, w\rangle |\ B \text{ is a NFA and } B \text{ accepts } w\}$

Is this decidable?

We could decide to design a TM that simulates the NFA on w. However, we can also convert the NFA into a DFA using the construction we designed to prove equivalence.

Theorem: A_{NFA} is decidable

Proof: Give TM $D_{\text{A-NFA}}$ that decides A_{NFA}.

$\boldsymbol{D_{\text{A-NFA}}}$ = "On input $\langle B, w\rangle$

1. Convert NFA B to equivalent DFA B'.
2. Run TM $\boldsymbol{D_{\text{A-DFA}}}$ on input $\langle B', w\rangle$. [Recall that $D_{\text{A-DFA}}$ decides A_{DFA}]
3. *Accept* if $D_{\text{A-DFA}}$ accepts.
   Reject if not."

- **New element:** Use conversion construction and previously constructed TM as a **subroutine**.

Generation Problem for Regular Expressions

- The **generation problem** for regular expressions is that of determining whether a reg. expr. generates a given string.

Let $A_{\text{REX}} = \{\langle R, w\rangle |$ R is a Reg. Expr. and R generates $w\}$

Is this decidable?

Theorem: A_{REX} is decidable

Proof: Give TM $D_{\text{A}-\text{REX}}$ that decides A_{REX}.

$\boldsymbol{D_{\text{A}-\text{REX}}}$ = "On input $\langle R, w\rangle$

1. Convert Reg. Expr. R to an equivalent NFA N.
2. Run TM $\boldsymbol{D_{\text{A}-\text{NFA}}}$ on input $\langle N, w\rangle$. [Recall that $D_{\text{A}-\text{NFA}}$ decides A_{NFA}]
3. *Accept* if $D_{\text{A}-\text{NFA}}$ accepts.
   Reject if not."

Summary: It is equivalent to decide DFAs, NFAs, or regular expressions because the TM can convert one form of encoding into another.

Emptiness Problem for DFAs

- The **emptiness problem** for DFAs is that of determining whether or not a DFA accepts or not any string at all.

Let $E_{\text{DFA}} = \{\langle B \rangle | \ B \text{ is a DFA and } L(B) = \emptyset\}$

Is this decidable?

Can we try the DFA on all possible strings?

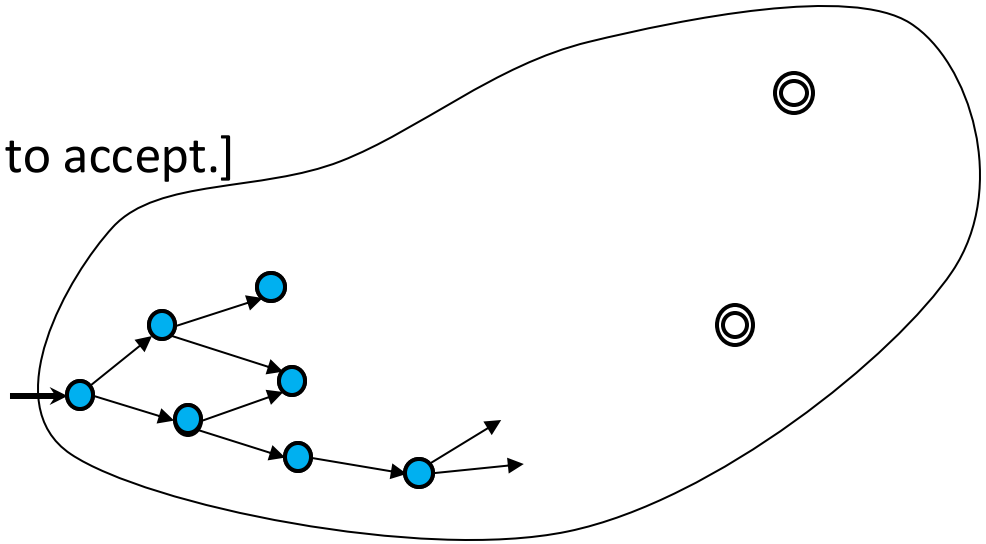
Theorem: E_{DFA} is decidable

Proof: Give TM $D_{\text{E-DFA}}$ that decides E_{DFA} .

$\boldsymbol{D_{\text{E-DFA}}}$ = "On input $\langle B \rangle$ [IDEA: Check for a path from start to accept.]

1. Mark start state.
2. Repeat until no new state is marked:

   Mark every state that has an incoming arrow from a previously marked state.
3. *Accept* if no accept state is marked.
   Reject if some accept state is marked."



Equivalence problem for DFAs

- The **equivalence problem** for DFAs is that of determining whether or not two DFAs recognize the same language.

Let $EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}$

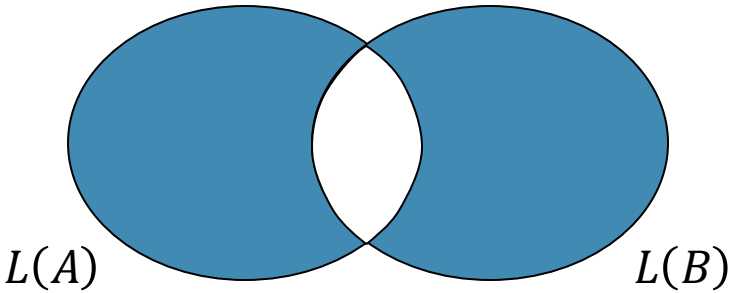
Is it decidable?

Theorem: EQ_{DFA} is decidable

Proof: Give TM $D_{\text{EQ}-\text{DFA}}$ that decides EQ_{DFA}.

$\boldsymbol{D_{\text{EQ}-\text{DFA}}}$ = "On input $\langle A, B\rangle$ [IDEA: Make DFA C that accepts w where A and B disagree.]

1. Construct DFA C where $L(C) = \left(L(A) \cap \overline{L(B)}\right) \cup \left(\overline{L(A)} \cap L(B)\right)$.
2. Run $\boldsymbol{D_{\text{E}-\text{DFA}}}$ on $\langle C\rangle$.
3. *Accept* if $D_{\text{E}-\text{DFA}}$ accepts.
   Reject if $D_{\text{E}-\text{DFA}}$ rejects."



Symmetric difference

Generating Problem for CFGs

- The **generating problem** for CFGs is that of determining whether a CFG generates a given string.
- This problem is related to that of recognising and compiling programs in a programming language.

Let $A_{\text{CFG}} = \{\langle G, w\rangle \mid G \text{ is a CFG and } w \in L(G)\}$

Is it decidable?

We cannot try all the derivations as these may be infinite (If G does not accept w, the TM would never halt)

Theorem: A_{CFG} is decidable

Proof: Give TM $D_{\text{A-CFG}}$ that decides A_{CFG}.

$\boldsymbol{D_{\text{A-CFG}}}$ = "On input $\langle G, w\rangle$

1. Convert G into CNF.
2. Try all derivations of length $2|w| - 1$.
3. *Accept* if any generate w.
   Reject if not.

Recall Chomsky Normal Form (CNF) only allows rules:

A → BC

B → b

Lemma 1: Can convert every CFG into CNF (shown)

Lemma 2: If H is in CNF and $w \in L(H)$ then every derivation of w has exactly $2|w| - 1$ steps.
Proof: exercise.

Check-in 9.2

Can we conclude that A_{PDA} is decidable?

a) Yes.

b) No, PDAs may be nondeterministic.

c) No, PDAs may not halt.

Check-in 9.2

Context Free Language Decidability

Let A be a CFL.

Is it decidable?

Bad Idea:

1. Convert a PDA for A into a TM (Simulating a stack on a TM is easy)

2. If the PDA is non-deterministic, we design a NTM and convert to a det. TM

Problem: The PDA may loop (reading and writing on the stack without ever halting): the simulating TM may also have some branches that never halt. The TM may not be a **decider**.

Corollary: Every CFL is decidable.

Proof: Let A be a CFL, generated by CFG G.

Construct TM M_G = "on input w

1. Run $\boldsymbol{D_{\text{A−CFG}}}$ on $\langle G, w \rangle$.
2. *Accept* if $D_{\text{A−CFG}}$ accepts
   Reject if it rejects."

Check-in 9.3

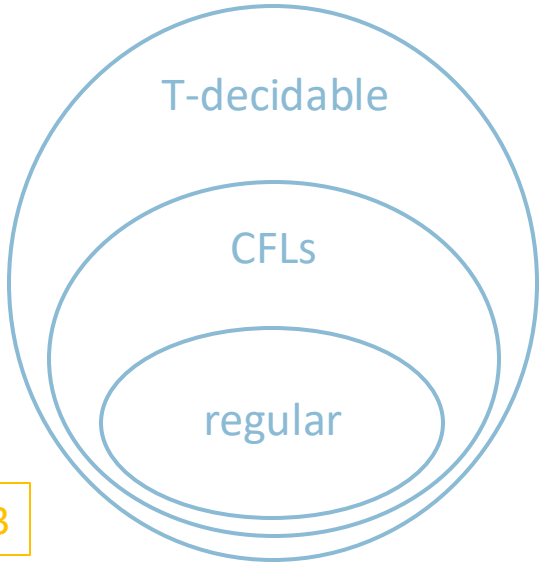
Are Regular Languages decidable?

a) Yes.

b) No.

c) I don't know.

Summary



Check-in 9.3

Emptiness Problem for CFGs

- The **emptiness problem** for CFGs is that of determining whether or not a CFG generates any string at all.

Let $E_{\text{CFG}} = \{\langle G \rangle | \ G \text{ is a CFG and } L(G) = \emptyset \}$

Is it decidable?

Theorem: E_{CFG} is decidable

Proof:

$\boldsymbol{D}_{\text{E-CFG}}$ = "On input $\langle G \rangle$ [IDEA: work backwards from terminals]

1. Mark all occurrences of terminals in G.
2. Repeat until no new variables are marked
   Mark all occurrences of variable A if
   $\text{A} \rightarrow \text{B}_1\text{B}_2 \cdots \text{B}_k$ is a rule and all B_i were already marked.
3. *Reject* if the start variable is marked.
   Accept if not."

S → RTa
R → Tb
T → a

Equivalence Problem for CFGs

- The **equivalence problem** for CFGs is that of determining whether or not two CFGs generate the same language.

Let $EQ_{\text{CFG}} = \{\langle G, H\rangle \mid G, H \text{ are CFGs and } L(G) = L(H)\}$

Is it decidable?

1) Try all strings?
2) Use the closure constructions we used for DFAs?
   - The class of CFL is not closed under intersection (last lecture) nor complementation (Lab exercise 7.3)

Theorem: EQ_{CFG} is NOT decidable

Proof: Next lecture.

Check-in 9.3

Why can't we use the same technique we used to show EQ_{DFA} is decidable to show that EQ_{CFG} is decidable?

a) Because CFGs are generators and DFAs are recognizers.
b) Because CFLs are closed under union.
c) Because CFLs are not closed under complementation and intersection.

Check-in 9.3

Acceptance Problem for TMs

- The **acceptance problem** for a TM is that of determining whether a Turing machine accepts a given string.

Let $A_{\text{TM}} = \{\langle M, w\rangle | \ M \text{ is a TM and } M \text{ accepts } w\}$

Is this decidable?

Theorem: A_{TM} is not decidable

Proof: Next lecture.

Is it recognizable?

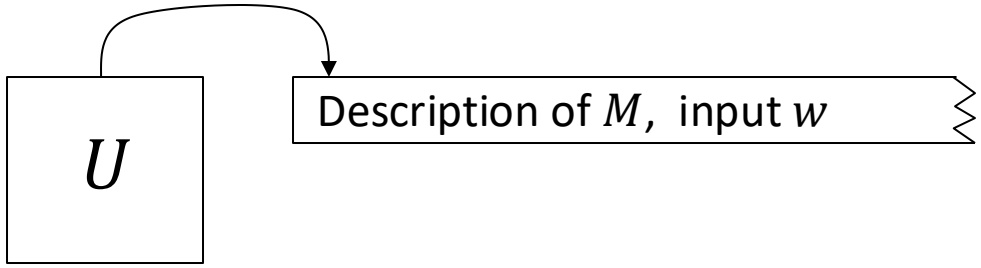
Theorem: A_{TM} is T-recognizable

Proof: The following TM U recognizes A_{TM}

$U =$ "On input $\langle M, w\rangle$

1. Simulate M on input w.
2. *Accept* if M halts and accepts.
3. *Reject* if M halts and rejects.
4. ~~*Reject* if M never halts."~~ Not a legal TM action.

Turing's original "Universal Computing Machine"
(capable of simulating another TM from the description of that machine)



Von Neumann said U inspired the concept of a stored program computer.

Quick review of today

1. We showed the decidability of various problems about automata and grammars:
   A_{DFA} , A_{NFA} , E_{DFA} , EQ_{DFA} , A_{CFG} , E_{CFG}
2. We showed that A_{TM} is T-recognizable.

Summary